

Autonomous Delivery Robot: Q-Learning in a Grid-World Environment

Project Documentation

1. Introduction

This project simulates an autonomous delivery robot operating within a stylized 10x10 urban grid environment. The core objective is to employ **Reinforcement Learning (RL)**—specifically **Tabular Q-Learning**—to teach the robot an optimal policy for navigating from a central depot to dynamically assigned pickup locations, and subsequently to target dropoff destinations, while avoiding obstacles (buildings).

Rather than relying on deterministic pathfinding algorithms (such as A* or Dijkstra's), this project demonstrates how an agent can iteratively learn spatial navigation and task completion through trial and error, guided purely by an engineered reward function.

2. Environment Architecture (**DeliveryBot-v0**)

The simulation environment is implemented as a custom **Gymnasium** environment (`delivery_env.py`).

2.1 Spatial Dynamics

- **Grid:** 10x10 discrete grid.
- **Obstacles:** 16 impassable building cells configured in a block layout.
- **Locations of Interest:**
 - **Depot:** Fixed at `(0, 0)`. The robot always begins its episode here.
 - **Pickup Nodes:** 4 possible locations `[(1,1), (1,8), (8,1), (8,8)]`.
 - **Dropoff Nodes:** 4 possible locations `[(0,5), (5,0), (5,9), (9,5)]`.

At the start of each episode, one pickup node and one dropoff node are randomly selected.

2.2 State Space

The environment must provide a Fully Observable Markov Decision Process (MDP). A naive state (just coordinates) is insufficient because the optimal action depends on the *current assigned mission*. The discrete state index encodes:

1. Robot X-coordinate (0-9)
2. Robot Y-coordinate (0-9)
3. Carrying Package Status (Boolean 0/1)
4. Active Pickup Location Index (0-3)
5. Active Dropoff Location Index (0-3)

Total Theoretical State Space Size: $10 * 10 * 2 * 4 * 4 = 3,200$ discrete states.

2.3 Action Space

The agent can select from a discrete action space of 6 potential actions:

- 0: Move Up
- 1: Move Down
- 2: Move Left
- 3: Move Right
- 4: Pick up Package
- 5: Drop off Package

2.4 Reward Function

The reward structure enforces efficiency, safety, and task completion:

- **Valid Step (-1):** Applied on standard movement to encourage the shortest path.
- **Illegal Action (-10):** Applied for collision with buildings, invalid pickups (wrong location or already carrying), or invalid dropoffs.
- **Successful Pickup (+20):** Intermediate positive reinforcement for reaching the first milestone.
- **Successful Dropoff (+100):** Terminal reward for completing the assigned delivery job.

3. Q-Learning Algorithm

The learning logic is executed in `delivery_qlearning.py`. We utilize a Tabular Q-learning algorithm, updating the agent's action-value estimations according to the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

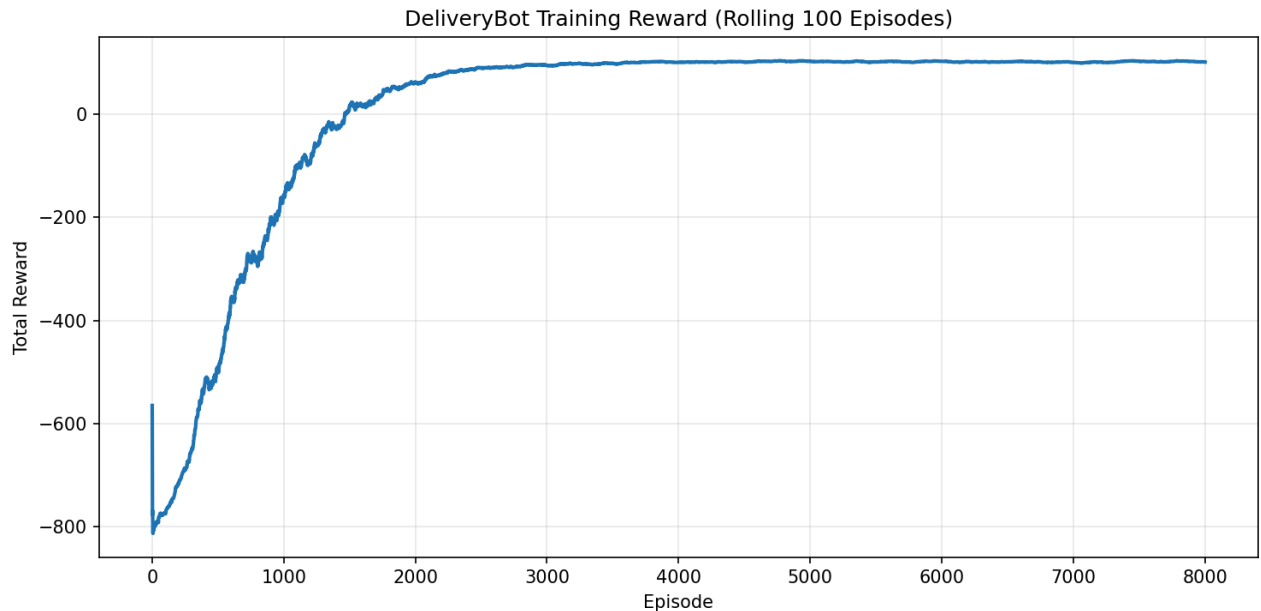
3.1 Hyperparameters

- **Learning Rate (α):** 0.1 — Controls the extent to which newly acquired information overrides old information.
- **Discount Factor (γ):** 0.99 — Represents the importance of future rewards. A value near 1.0 makes the agent heavily weight long-term success over immediate gratification.
- **Exploration Strategy:** ϵ -greedy approach.
 - **EPSILON_START:** 1.0 (100% random exploration initially).
 - **EPSILON_DECAY:** 0.999 (Multiplicative decay per episode).
 - **EPSILON_MIN:** 0.01 (Minimum 1% chance to explore to prevent getting stuck in local optima).
- **Training Duration:** 8,000 episodes, capped at 150 steps per episode.

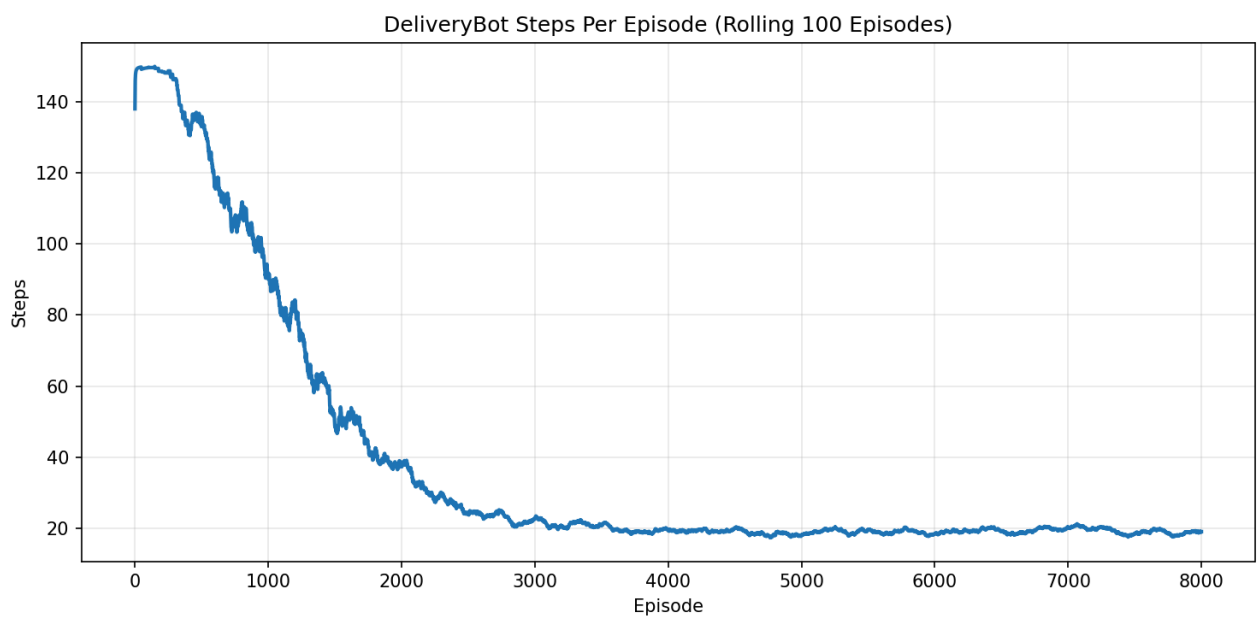
4. Evaluation and Metrics

During training, the system logs the agent's performance, producing rolling-average visualizations.

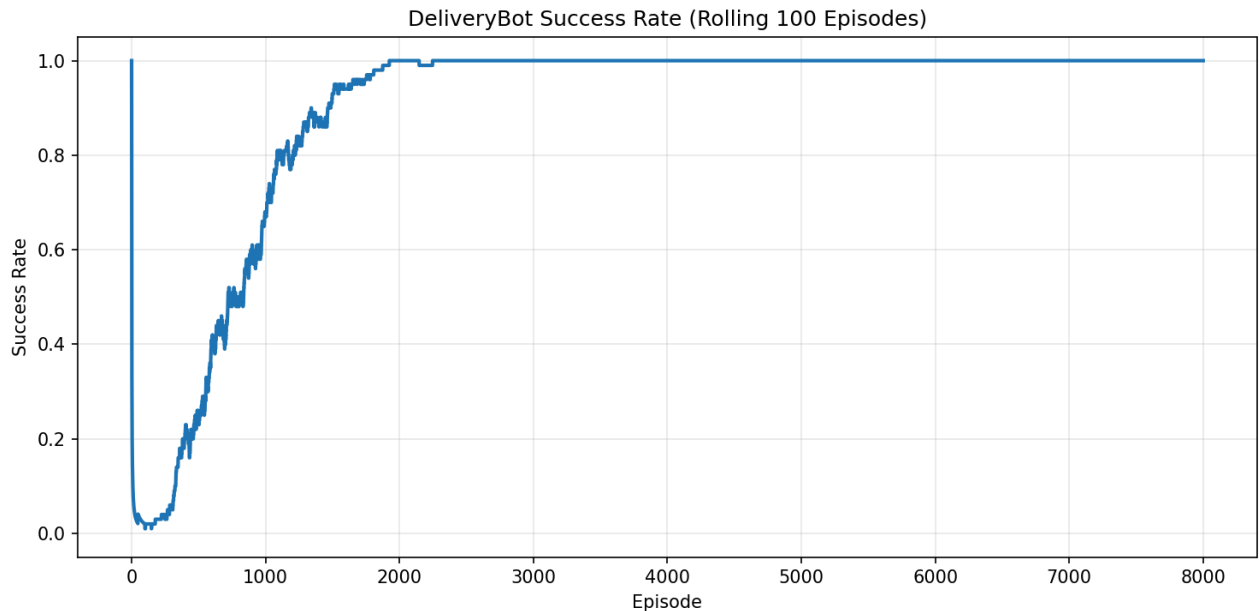
- **rewards.png:** Illustrates the agent's cumulative reward per episode. This metric rises sharply as the agent stops hitting obstacles and learns to complete the deliveries.



- **steps.png**: Depicts the number of steps required to terminate an episode. Over time, this curve drops as the agent learns the most direct, valid paths.



- **accuracy.png**: Shows the delivery success rate (percentage of episodes where the robot successfully drops off the package).



5. Visualizer

The environment features a comprehensive **pygame**-based rendering engine. It maps the internal matrix representation to visual assets (roads, sidewalks, crosswalks, dynamic shadows, glowing streetlights, and an animated robot). The visualizer provides an intuitive way to verify the resulting optimal policy, proving that the robot dynamically avoids buildings and routes properly toward varied randomized goals.

6. Setup & Execution Instructions

Prerequisites

- Python 3.10+
- Dependencies: **gymnasium**, **numpy**, **matplotlib**, **pygame**

Running the Project

The environment requires an active python virtual environment containing the necessary dependencies (located in `./venv`).

1. Standard Run (Train + Demo):

```
venv\Scripts\python.exe delivery_qlearning.py
```

This script will train the model for 8,000 episodes, export the learning plots, save the policy to **delivery_qtable.npy**, and spawn a Pygame window showing 10 simulated test runs.

2. Headless Training (No Pygame Demo):

```
venv\Scripts\python.exe delivery_qlearning.py --skip-demo
```

3. Run Demo from Saved Policy:

```
venv\Scripts\python.exe delivery_qlearning.py --demo-only
```

4. CLI Arguments:

- `--episodes <int>`: Number of training episodes.
- `--demo-episodes <int>`: Number of test runs to render.
- `--demo-delay-ms <int>`: Time step delay for the visualization.

7. Conclusion

This simulation successfully models how reinforcement learning can be practically applied to autonomous navigation. The Q-learning agent effectively maps an unknown, bounded state space, determining an optimal, generalized routing policy capable of addressing any combination of randomized task variables without explicitly programmed pathfinding heuristics.